

```

#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include "inc/hw_memmap.h"
#include "driverlib/adc.h"
#include "driverlib/gpio.h"
#include "driverlib/pin_map.h"
#include "driverlib/sysctl.h"
#include "driverlib/uart.h"
#include "utils/uartstdio.h"
#include "inc/tm4c123gh6pm.h"
#include "driverlib/interrupt.h"
#include "driverlib/systick.h"

#define RED    GPIO_PIN_1
#define BLUE   GPIO_PIN_2
#define SW1    GPIO_PIN_4

//Prototypes
void InitConsole(void);
void ADCInit (void);
void SysTick_Handler(void);
void switchInterrupt(void);
uint32_t getCelsiusValue(void);
uint32_t convertToFahrenheit (uint32_t TempC);
void PrintTemps (uint32_t TempC, uint32_t TempF);

// Global variables
uint8_t hours, minutes, seconds, remaining_milliseconds;

//Determines the sample rate of the ADC
uint32_t clkscalvalue = 6;

uint32_t ui32TempValueF;

volatile uint32_t timer_count = 0;
volatile uint32_t ui32TempValueC;

void delay(int tics){
    int i;
    for(i=0; i<tics; i++){
    }
}

int main(void)
{
    // local variables to Main
    //uint32_t ui32TempValueC;

    // System clock initialization
    SysCtlClockSet(SYSCTL_SYSDIV_2_5|SYSCTL_USE_PLL|SYSCTL_XTAL_16MHZ|
SYSCTL_OSC_MAIN);

    // GPIO initialization
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF); //ENABLE PORT F
    GPIOPinTypeGPIOOutput(GPIO_PORTF_BASE,BLUE|RED); //RED AND BLUE LED AS OUTPUTS
    GPIOPinTypeGPIOInput(GPIO_PORTF_BASE,SW1); //SWITCH AS INPUT

    GPIOPadConfigSet(GPIO_PORTF_BASE, SW1, GPIO_STRENGTH_4MA, GPIO_PIN_TYPE_STD_WPU);

```

```

//WEAK PULL UP FOR SWITCH

// GPIO interrupt initialization for push button
GPIOIntTypeSet(GPIO_PORTF_BASE, ~SW1, GPIO_RISING_EDGE); //TRIGGER INTERRUPT ON
RISING EDGE
GPIOIntRegister(GPIO_PORTF_BASE, switchInterrupt); // REGISTER SWITCH INTERRUPT
GPIOIntEnable(GPIO_PORTF_BASE, SW1); // ENABLE INTERRUPTS ON THE SWITCH

// SysTick timer initialization
SysTickEnable();

//80MHz clock generates 8000000 ticks per 10ms

SysTickPeriodSet(SysCtlClockGet()/1000); // 1 ms
SysTickIntRegister(&SysTick_Handler);
SysTickIntEnable(); // interrupt enable

// UART initialization
InitConsole();

// ADC initialization
ADCInit();

// Display the setup on the console.
UARTprintf("ECE505 Lab 4: ADC and Interrupts\n");
UARTprintf("*****\n");
UARTprintf("Analog Input: Internal Temperature Sensor\n");

UARTprintf("Initialization Complete...\n");

// Sample the temperature sensor forever. Display the value on the
// console.
while(1)
{
    //It's alot more efficient to put the celsius readings into a function
    //ui32TempValueC = getCelsiusValue();

    //Convert to Fahrenheit
    ui32TempValueF = convertToFahrenheit(ui32TempValueC);

    // Display the temperature value on the console.
    PrintTemps(ui32TempValueC, ui32TempValueF);

    //
    // This function provides a means of generating a constant length
    // delay. The function delay (in cycles) = 3 * parameter. Delay
    // 250ms arbitrarily.
    //
    SysCtlDelay(SysCtlClockGet() / clkscalvalue);
}
} //end main

void switchInterrupt(void)
{
    unsigned char blue_led_status;

    GPIO_PORTF_ICR_R = 0x10; // acknowledge flag4

```

```

    blue_led_status = GPIOPinRead(GPIO_PORTF_BASE, RED);

    if(blue_led_status == 0)
        GPIOPinWrite(GPIO_PORTF_BASE, BLUE, BLUE);
    else
        GPIOPinWrite(GPIO_PORTF_BASE, BLUE, 0x00);

    delay(1000000); // delay for "debouncing"
}

void SysTick_Handler(void)
{
    timer_count++;
    ui32TempValueC = getCelsiusValue();

    unsigned char red_led_status;

    //GPIO_PORTF_DATA_R ^=0x02;

    red_led_status = GPIOPinRead(GPIO_PORTF_BASE, RED);

    if(red_led_status == 0)
        GPIOPinWrite(GPIO_PORTF_BASE, RED, RED);
    else
        GPIOPinWrite(GPIO_PORTF_BASE, RED, 0x00);
}

uint32_t convertToFahrenheit (uint32_t TempC)
{
    //Fill out this function
    uint32_t TempF=0;
    //uint32_t temporary = TempC * (1.8);
    TempF = ((TempC * (1.8)) + 32);

    return TempF;
}

uint32_t getCelsiusValue(){

    // This array is used for storing the data read from the ADC FIFO. It
    // must be as large as the FIFO for the sequencer in use.
    uint32_t ui32ADC0Value[4];

    // These variables are used to store the temperature conversions for
    // AVG, Celsius and Fahrenheit.
    uint32_t ui32TempAvg;

    // Trigger the ADC conversion.
    ADCProcessorTrigger(ADC0_BASE, 1);

    // Wait for conversion to be completed.
    while(!ADCIntStatus(ADC0_BASE, 1, false))
    {
    }
}

```

```

    // Clear the ADC interrupt flag.
    ADCIntClear(ADC0_BASE, 1);

    // Read ADC Values.
    ADCSequenceDataGet(ADC0_BASE, 1, ui32ADC0Value);

    //Average the 4 Samples
    ui32TempAvg = (ui32ADC0Value[0] + ui32ADC0Value[1] + ui32ADC0Value[2] +
    ui32ADC0Value[3] + 2)/4;

    //Convert Raw Data to Temp Celsius
    return (1475 - ((2475 * ui32TempAvg)) / 4096)/10;
}

//This function prints the two passed temperatures to the console in an
//easy human readable format.
void PrintTemps (uint32_t TempC, uint32_t TempF)
{
    //SysTickIntDisable();
    seconds = timer_count/1000;
    remaining_milliseconds = timer_count % 1000;

    minutes = seconds/60;
    seconds = seconds % 60;

    hours = minutes/60;
    minutes = minutes % 60;

    //UARTprintf("%2d:%2d:%2d:%2d\t%3d*C\t%3d*F\n", hours, minutes, seconds,
    remaining_milliseconds, TempC, TempF);
    UARTprintf("%2d:%2d:%2d:%2d\t%3d\t%3d\n", hours, minutes, seconds,
    remaining_milliseconds, TempC, TempF);

    //SysTickEnable();
    //UARTprintf("%3d*C\t%3d*F\n", TempC, TempF);
}

//PROVIDED FUNCTIONS. NO NEED TO MODIFY
//*****
//
// This function sets up UART0 to be used for a console to display information
// as the lab is running.
//
//*****
void InitConsole(void)
{
    // Enable GPIO port A which is used for UART0 pins.
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);

    // Configure the pin muxing for UART0 functions on port A0 and A1.
    GPIOPinConfigure(GPIO_PA0_U0RX);
    GPIOPinConfigure(GPIO_PA1_U0TX);

    // Enable UART0 so that we can configure the clock.
    SysCtlPeripheralEnable(SYSCTL_PERIPH_UART0);

    // Use the internal 16MHz oscillator as the UART clock source.
    UARTClockSourceSet(UART0_BASE, UART_CLOCK_PIOSC);

```

```

        // Select the alternate (UART) function for these pins.
        GPIOPinTypeUART(GPIO_PORTA_BASE, GPIO_PIN_0 | GPIO_PIN_1);

        // Initialize the UART for console I/O. 9600 BAUD
        UARTStdioConfig(0, 9600, 16000000);
    }

//*****
//
// This function configures the ADC0 Peripheral for EE383LABS 4 and 5
// Configuration:
// ADC0, Sequence 1(4 Samples)
//
//*****
void ADCInit(void)
{
    //Enable PORTE for analog input
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);
    while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOE))
    {}
    //Configure PORTE pins for analog input
    //PE1 analog input CH2
    GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_3);

    // The ADC0 peripheral must be enabled for use.
    SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);
    while(!SysCtlPeripheralReady(SYSCTL_PERIPH_ADC0))
    {}

    // Enable sample sequence 1 with a processor signal trigger. Sequence 1
    // will do a four samples when the processor sends a signal to start the
    // conversion. Each ADC module has 4 programmable sequences, sequence 0
    // to sequence 3. This lab is arbitrarily using sequence 1.
    ADCSequenceConfigure(ADC0_BASE, 1, ADC_TRIGGER_PROCESSOR, 0);

    // Configure ADC0 on sequence 1. Sample the temperature sensor
    // (ADC_CTL_TS) and configure the interrupt flag (ADC_CTL_IE) to be set
    // when the sample is done. Tell the ADC logic that this is the last
    // conversion on sequence 3 (ADC_CTL_END). Sequence 1 has 4
    // programmable steps. Sequence 1 and 2 have 4 steps, and sequence 0 has
    // 8 programmable steps. Sequence3 has 1 step. For more information on the
    // ADC sequences and steps, reference the datasheet.

    ADCSequenceStepConfigure(ADC0_BASE, 1, 0, ADC_CTL_TS);
    ADCSequenceStepConfigure(ADC0_BASE, 1, 1, ADC_CTL_TS);
    ADCSequenceStepConfigure(ADC0_BASE, 1, 2, ADC_CTL_TS);
    ADCSequenceStepConfigure(ADC0_BASE, 1, 3, ADC_CTL_TS|ADC_CTL_IE|ADC_CTL_END);

    // ADCSequenceStepConfigure(ADC0_BASE, 1, 0, ADC_CTL_CH2);
    // ADCSequenceStepConfigure(ADC0_BASE, 1, 1, ADC_CTL_CH2);
    // ADCSequenceStepConfigure(ADC0_BASE, 1, 2, ADC_CTL_CH2);
    // ADCSequenceStepConfigure(ADC0_BASE, 1, 3, ADC_CTL_CH2|ADC_CTL_IE|
    ADC_CTL_END);

    // Since sample sequence 1 is now configured, it must be enabled.
    ADCSequenceEnable(ADC0_BASE, 1);
}

```

```
    // Clear the interrupt status flag. This is done to make sure the
    // interrupt flag is cleared before we sample.
    ADCIntClear(ADC0_BASE, 1);
}

/*void SysTick_IRQ_Handler(){
    timer_count++;
}*/

//EOF
```